Autonomous Driving: Integration of Segmentation and Depth Camera in a Curriculum Learning Approach

Silvio Barra, *Member, IEEE*, Lucia Cimmino, *Member, IEEE*, Vincenzo Loia, *Senior Member, IEEE*, Michele Nappi, *Senior Member, IEEE*, and Matteo Polsinelli, *Member, IEEE*

***Abstract*—Autonomous driving (AD) entails vehicles that can perceive their surroundings and navigate without human intervention. This involves utilising a combination of sensors and algorithms to recognize obstacles, interpret traffic signals, and make driving decisions. While AD holds promise for transforming transportation by enhancing safety, reducing congestion, minimising pollution, and optimising efficiency, it poses technical challenges also. This work extends a novel approach to building an autonomous vehicle agent using deep reinforcement learning (DRL) with proximal policy optimisation (PPO) to navigate urban environments simulated by the CAR learning to act (CARLA) Simulator. The agent aims to maintain lane integrity and avoid collisions, even in adverse weather conditions. The proposed architecture integrates a 180-degree environmental view and various multimodal data inputs (RGB, segmentation, and depth camera inputs), extensively tested through experimentation. Notably, the integration of segmentation and depth data results in a 13% reduction in the collision rate, with the proposed agent achieving a total reward of 2510. This approach demonstrates significant progress over the previous framework, showcasing improved obstacle detection and collision avoidance accuracy. Moreover, these findings contribute to ongoing autonomous vehicle research, offering insights into effective strategies for developing robust and dependable driving agents capable of navigating urban environments and interacting with road infrastructure, contributing to advancements in Augmented Intelligence of Things (AIoT)-enabled AD.**

***Index Terms*—Autonomous driving (AD), curriculum learning (CL), deep reinforcement learning (DRL), object segmentation, proximal policy optimisation (PPO).**

Received 26 August 2024; revised 4 November 2024, 30 November 2024, 30 April 2025, and 28 May 2025; accepted 3 June 2025. Date of publication 17 June 2025; date of current version 25 August 2025. This work was supported in part by the Project IDA included in the Spoke 2—Misinformation and Fakes of the Research and Innovation Program under Grant PE00000014, and in part by the "SEcurity and RIghts in the CyberSpace (SERICS)", under the National Recovery and Resilience Plan, Mission 4 "Education and Research" - Component 2 "From Research to Enterprise" - Investment 1.3, funded by the European Union - NextGenerationEU. *(Corresponding author: Lucia Cimmino.)*

Silvio Barra is with the Department of Information Technology and Electrical Engineering, University of Naples Federico II, 80138 Napoli, Italy.

Lucia Cimmino and Michele Nappi are with the Department of Computer Science, University of Salerno, 84084 Salerno, Italy (e-mail: lcimmino@unisa.it).

Vincenzo Loia and Matteo Polsinelli are with the Department of Management and Innovation Systems, University of Salerno, 84084 Salerno, Italy.

Digital Object Identifier 10.1109/JIOT.2025.3579862

I. Introduction

AUTONOMOUS driving (AD), also known as self-driving or driverless technology, refers to vehicles capable of sensing their environment and driving without human intervention [1]. To achieve these results, the vehicles utilize a combination of sensors, cameras, radar, light detection and ranging (LiDAR), global positioning system (GPS), and algorithms to perceive the surrounding environment, identify obstacles, interpret traffic signs and signals, and make driving decisions [2].

AD has the potential to deeply change transportation, improving safety [3], [4], reducing traffic congestion [5], reducing pollution, and enhancing efficiency in transportation systems. However, it also presents various technical challenges that need to be addressed. These include issues related to safety, liability, and robustness of the algorithm that makes the decisions [6]. The society of automotive engineers (SAE) identified five different levels of AD, starting from Level 0 (no automation) to Level 5 (full automation) for which the vehicle can perform all driving tasks under all conditions without human intervention. Several variables could prevent Autonomous Vehicles (AVs) from working effectively: changing weather conditions, illumination, sensor failure, occlusions, etc.

The standard way to tackle this problem is to develop complex simulation environments that correspond to the most challenging situations and test the algorithms on these scenarios. The acquisition of large datasets of driving instances provided by human experts could be difficult, expensive, time-consuming, and biased according to the preferences of the drivers themselves. In addition, the test of autonomous vehicles in the real world involves significant safety and ethical considerations and poses risks to human life and property. For this reason, sophisticated and realistic simulators for AD have been developed [7], [8] that provide a safe and ethical platform to evaluate and validate the performance of the autonomous vehicle agent before deploying it in real-world environments. These software applications implement several digital assets (urban layouts, buildings, vehicles) and commonly used sensors to interact with the virtual environment.

This has facilitated the development of artificial intelligence (AI)-based methods, in particular, deep reinforcement learning (DRL) [9], [10], [11], [12] methods.

When AI is combined with Internet of Things (IoT), a new technology called Augmented Intelligence of Things (AIoT)

© 2025 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

emerges with the goal of enhancing the capabilities of IoT devices through intelligent algorithms and data analytics [5]. AIoT enables IoT devices to gather, process, and analyse vast amounts of data in real-time, leading to more intelligent decision-making and automation. Combining AI with IoT empowers devices to adapt, learn, and evolve, ultimately driving innovation and efficiency in interconnected systems, such as those found in autonomous vehicles [13].

However, for very complex scenarios, even with very long training times, the agents could not converge to high-quality solutions and consequentially poor driving performance could still occur.

A promising solution is curriculum learning (CV) [14], [15], [16]. With a CV strategy, the DRL agent is trained in simpler scenarios that gradually increase their complexity (increasing the number of other vehicles on the road, introducing pedestrians that cross the road, changing the weather from sunny to rainy, etc.) until the agent is capable also to handle the most complex and realistic ones.

To further increase the performance of the AD agents, multiple inputs from the sensors commonly available for the AD should be used [17], [18], [19]. Multimodal data from different types of sensors (RGB, depth, LiDAR, etc) provide a more comprehensive view of a situation by incorporating different types of information [20]. This can lead to a more nuanced understanding of complex scenarios, and enhance the performance and the robustness of the agent. Although many multimodal fusion detection algorithms exist, how to effectively fuse multimodal data is still under investigation [19].

The proposed work has been deeply inspired by [21]. In that work, the authors presented an agent trained using DLR and CL strategy. The agent's performances are consistent in all the testing scenarios.

The primary limitation of the method lies in its difficulty in accurately detecting obstacles, which results in a high rate of collisions. This issue can be attributed, in part, to the exclusive reliance on RGB cameras, without consideration of alternative camera types or the integration of multimodal data. To address this limitation, the present study offers a comprehensive exploration of the use of multimodal cameras. In addition to RGB cameras, the experimental setup incorporates depth and segmentation cameras.

While LiDAR and thermal imaging offer compelling capabilities, our decision to focus on segmentation maps and depth images was driven by a deliberate emphasis on affordability, energy efficiency, and ease of deploymentkey considerations for scalable, real-world applications where lightweight and accessible solutions are essential.

The images generated by these cameras were used either independently, concatenated, or in separate streams, modifying the original model proposed by Anzalone et al. [21]. Specifically, a dedicated neural network branch was introduced and tested to accommodate the multimodal inputs. The integration of multiple sensors enhances the system's robustness and reliability by providing redundant data sources that capture different perspectives. This redundancy enables the system to mitigate potential issues arising from individual sensor failures or discrepancies, as sensor fusion techniques combine the diverse inputs to ensure accurate and consistent performance.

The contributions of this article are the following.

1) In addition to the standard vehicle view, which comprises three concatenated cameras capturing the left, centre, and right sides of the car, segmented cameras are incorporated into the training process. Segmentation enables more accurate localisation of potential obstacles, enhancing the comprehension of the surrounding environment.
2) A depth camera is integrated into the model, augmenting the vehicle's capability to perceive the proximity of objects.
3) Various combined approaches are explored and evaluated during the training process to analyse the effective contribution of the segmentation and depth cameras.
4) Finally, an analysis of the energy consumption required by the proposed method has been performed.

In contrast to [21], which focuses exclusively on CL-based approach with RGB input, the proposed approach explores alternative sensor modalities and evaluates their fusion through a streamlined, single-backbone design. This enables better understanding of the scene while reducing training complexity and computational cost.

All agents were evaluated using six metrics: *Collision rate*, *Similarity*, *Waypoint distance*, *Total reward*, and *Timesteps*, across different weather conditions and scenarios.

The obtained results indicates that the combination of depth and segmentation cameras yields superior performance compared to the method proposed by Anzalone et al. [21] across nearly all metrics and scenarios. In terms of energy consumption, the results are comparable to and slightly better than, those reported in [21].

II. Related Works

The architectures for autonomous vehicles can be separated into different categories based on different methodologies [1].

Between them, End-to-end driving is one of the most promising. End-to-end stands for the direct mapping from raw sensor data into trajectory points or control signals. In this specific context, End-to-end driving generates ego-motion directly from the raw data coming from the sensors, typically RGB cameras, Depth Cameras, and LiDAR. Because of its ability to extract task-specific policies, it has achieved great success in a variety of fields [22]

The end-to-end method can be divided into several categories based on the learning methods in particular imitation learning (IL) (a supervised method) and DRL (an unsupervised method).

A. Imitation Learning

IL is a machine learning paradigm where an agent learns a task by observing demonstrations provided by an expert [23]. IL for autonomous vehicles can be categorized into three main approaches [23]: 1) behavioral cloning (BC) [24]; 2) direct policy learning (DPL); and 3) inverse reinforcement learning (IRL) [25]. BC can lead to cutting-edge results even in new environments, where no specific training has been conducted. However, there are well-known limitations [26], such as generalisation problems due to the dynamic nature of objects in driving situations.

DPL builds upon BC by incorporating expert guidance during both training and runtime to improve policy performance [23]. Initially, DPL follows a similar methodology to BC by collecting a dataset of state-action pairs provided by an expert. However, DPL introduces an expert during runtime to evaluate and refine the policy, particularly in circumstances where the initial policy fails. This iterative process allows DPL to address the limitations of BC and enhance the policy's effectiveness across various scenarios.

Feature-based IRL operates under the assumption that an expert's reward function can be expressed as a linear combination of features. These features are specific to the task at hand and, in the context of AD, could encompass factors, such as distances to other cars or road markings [23].

The main limitation of IL methods is the necessity of a large dataset of driving instances provided by human experts. Acquiring these data can be difficult, expensive, and time-consuming. Furthermore, the collected data deeply depends on the preferences or habits of the drivers themselves. Consequentially, this could represent an issue regarding the generalizability of the method, which could be compromised if the dataset is intrinsically flawed (e.g., overfitting or uneven distribution) [27]. Finally, since the objective of IL methods is to acquire state-to-action mapping directly from the data when the agent is guided to an unknown state, predicting the correct behavior becomes a difficult challenge [28].

B. Deep Reinforcement Learning

In recent years, to address the limitations of IL methods, different approaches based on DRL have been proposed [11], [29].

One of the first attempts to apply DRL to AD is [30] in which the author trained their method to follow the lane of the road. Even if they also tested their method in a real-world driving scenario, the task is simple and did not consider, for example, other cars, pedestrians, and road intersections. DRL has shown advantages also in specific contexts of AD, such as its use at intersections or for overtaking manoeuvres on highways.

In [31], it addresses the complexity of intersections, which arises from the uncertainty in behaviors of surrounding vehicles and the different types of scenarios that can occur.

Other studies [32], [33] show the potential of DRL even in unsignalized intersections. These approaches could be extended to our specific case, not only at intersections but in every road segment.

In [34], an adaptive and optimal highway overtaking strategy based on DRL is proposed. In particular, a specific DRL algorithm, dueling deep Q-network (DDQN), is used to predict the vehicle's decision strategy.

The presented approaches [31], [32], [33], [34], confirm that DRL has the potential to bring improvements in AD. However, to improve the performance of AD agents, it is possible to combine DRL with CV [14]. CV is a technique inspired by human learning, where the difficulty of learning tasks is gradually increased over time.

In the context of AD tasks, CV could have several advantages: 1) starting with simpler tasks and gradually increasing complexity, AD systems can progressively develop the skills necessary to handle even more complex scenarios; 2) improve robustness and reliability; 3) better generalisation by exposing the agent to a diverse range of driving scenarios in a structured manner; 4) reducing training time while improving the performance of the agent.

In [16], the results demonstrate the algorithm was capable to adapt the scenario and make appropriate decisions based on the actions of other vehicles, thus contributing to the enhancement of AD at intersections.

In [31] the authors developed a method for intersection handling that combines DRL with CL. The aim was to train an algorithm that can autonomously learn to solve different types of intersections. The authors proposed a two-stage training strategy, in which the proposed method was first trained with a lightweight simulator. Then, during the second training step, the method was trained in a more realistic simulator. This has allowed the authors to save computational time.

A more complete study [15], in which the authors investigated the impact of different curricula on DRL for AD in a realistic driving simulator, was presented. They build an environment in which it is possible to tune the weather conditions and the complexity of the road to set several different curricula with increasing difficulty. The results provided in this article suggest that the agents trained using CL are better able to handle complex scenarios than the methods trained directly on the latter.

Similar results were obtained in [21], in which the authors combined the Proximal Policy Optimization algorithm (PPO) [35] with CL, evaluating various metrics and testing environments. The agent's performance is consistent in the various scenarios, demonstrating the method's validity. However, as the authors pointed out, the method has several weaknesses: the agent struggles to coordinate acceleration and breaking and to recognize obstacles resulting in collisions with obstacles in the environment. Since the authors have only tested as input for the method three RGB cameras positioned on top of the simulated vehicle, the use of other modalities could improve agent performance [36], [37].

Recent research also highlights the benefits of integrating CV with multimodal input strategies. For example, Jin et al. [38] demonstrated that combining visual and force sensors within a structured curriculum enabled more efficient policy acquisition in robotic assembly tasks. Although applied to a different domain, their findings support the broader applicability of CL in scenarios where agents must extract and integrate information from multiple complementary modalities.

The literature review indicates that DRL methods, when combined with computer vision, hold significant potential for achieving effective AD. However, the primary limitation of the reviewed approaches is their lack of robustness in complex real-world scenarios, such as intersections with high traffic or situations requiring simultaneous multiaction decision-making [39], [40], [41]. A potential solution to this challenge lies in the use of multimodal inputs, though this approach has been rarely considered to enhance the performance of existing methods. Moreover, a comprehensive and well-structured experimental evaluation is often lacking.

One possible reason for the limited exploration of multimodal inputs is the increased computational cost associated with integrating multiple sensors. Therefore, it is crucial to conduct studies that provide deeper insights into which combinations of multimodal inputs are most relevant and effective in addressing complex scenarios. Additionally, attention must be paid to the associated energy consumption, as the overhead introduced by processing multiple data streams can significantly impact system efficiency. This aspect, though often underestimated, is essential for the practical implementation of such systems.

III. Background

Reinforcement learning (RL) is a learning approach that addresses decision-making problems. It involves a software or physical agent learning to make optimal actions in an environment through trial and error, guided by positive or negative reward signals. In RL, the environment is represented as a Markov decision process (MDP), consisting of a state space (S), action space (A), transition model (P) for predicting state evolution, reward function (r), and (0, 1] ranged discount factor (γ). The agent perceives states (s) or observations (o) from the environment, and its actions are determined by a policy (π) that maps states to actions. In partially observable settings (POMDP), the agent conditions its actions on past observations ($o_(j : t)$). A deep neural network (DNN) is used as a policy network (π_θ), with learnable parameters (θ), to sample actions from a probability distribution. Continuous actions can be sampled from a Beta distribution, which is suitable for bounded continuous actions. The agent interacts with the environment by receiving an initial state (s_0) and executing actions (a_0) based on its policy. The environment responds with the next state (s_1) and an immediate reward (r_0). This interaction loop continues for multiple timesteps until a final state or a maximum timestep limit (T) is reached.

PPO [35] is a popular algorithm in the field of RL that addresses the problem of training policies for decision-making agents. PPO is designed to optimize policy parameters to maximise cumulative rewards in an environment.

In order to define how much an action a_i is better or worse than the average action for a specific state s, an advantage function is needed. An Advantage Function is defined as

$$A(s, a) = Q(s, a) - V(s) \tag{1}$$

in which $Q(s, a)$ is the action-value function and $V(s)$ is the state-value function. In particular, in the proposed approach, the generalized advantage estimation (GAE) [42] technique is used. GAE is a method used to estimate the advantages of different actions taken by an agent in an environment. Advantages represent the expected improvement of taking a particular action a_i compared to the average action a_ave in a given state. GAE method needs two hyperparameters: γ which is the discount factor, and λ which controls the balance between bias and variance in advantage estimation

$$A_t^{(\mathrm{GAE}(\gamma,\lambda)} = (1-\lambda)\Big(A_t^{(1)} + \lambda A_t^{(2)} + \lambda A_t^{(3)} + \cdots\Big). \tag{2}$$

IV. Proposed System

To address the limitations and enhance the model proposed by Anzalone et al. [21], a thorough analysis of the current state-of-the-art in AD was conducted. This analysis aimed to identify weaknesses in the algorithm employed and to devise innovative solutions for their mitigation.

The proposed system and the enhancements made to refine the method of Anzalone et al. [21] focused on improving the information derived from the agent's camera, aiming to minimize the increase in computational load. To boost the agent's comprehension of the environment, semantic segmentation of the camera view and depth camera information was incorporated. These approaches were applied while considering the time and resources required for the training process.

For the simulation of the environments, vehicles, and their interactions, the CAR Learning to Act (CARLA) simulator was utilized [43]. CARLA is an open-source driving simulator designed for research and development purposes in the field of AD. It provides open-access digital assets, such as urban layouts, buildings, and vehicles, which were developed for this purpose and are freely available for use. The platform allows for flexible customization of sensor suites and environmental conditions, making it highly adaptable for a range of research needs.It provides a realistic virtual environment for testing and training AD algorithms. CARLA supports synchronous execution for sensor data capture and vehicle control, enabling time-aligned data streams that are essential for DRL and real-time control strategies. It also offers various features, such as realistic urban environments, dynamic traffic scenarios, weather conditions, sensor simulation (including cameras, LiDARs, and radars), and the ability to customize scenarios for specific research needs. CARLA natively supports multiple sensors, including RGB, depth, and semantic segmentation cameras, LiDAR, radar, GPS, and inertial measurement unit (IMU), with configurable noise models to simulate real-world inaccuracies. This allows for controlled experimentation in multimodal sensor fusion and fault injection studies. Additionally, CARLA integrates with middleware, such as Robot Operating System (ROS) [44] and OpenDRIVE maps [45], enabling seamless extension to hybrid simulation platforms or digital twin pipelines.

Among the available simulators, CARLA is particularly distinguished by its ability to simulate high-fidelity sensory data, its sustained and proactive maintenance accompanied by regular updates, and its unique functionalities that are not currently offered by other open-source simulators [7].

A. CV Approach From Anzalone et al.

As mentioned before, this study aims to extend the curriculum learning (CL)-based methodology proposed by Anzalone et al., as detailed in their paper titled *"An end-to-end CL approach for AD scenarios"* [21]. The approach combines RL with CL techniques. Specifically, the training process is organized into five stages, each progressively increasing in difficulty.

Stage 1: The agent is evaluated in an uncrowded environment devoid of pedestrians, vehicles, or adverse weather conditions.

Stage 2: The CARLA Simulator introduces 50 randomly generated pedestrians into the town environment, posing challenges to the vehicle. No other vehicles or adverse weather conditions are present.

Stage 3: The agent is tested in an environment with 50 pedestrians, 50 vehicles and different weather conditions. The weather parameter is randomly set to one of the following values `[ClearSunset, ClearNoon, SoftRainSunset, SoftRainNoon, WetSunset, WetNoon, CloudyNoon]`.

Stage 4: Similar conditions to stage 3, but with data augmentation enabled.

Stage 5: Similar conditions to stage 4, but with an increased number of pedestrians and vehicles (200 pedestrians and 100 vehicles).

The five stages are executed sequentially, with each subsequent stage utilising the weights obtained from the preceding stage as input. This strategy was complemented with the PPO technique [35]. PPO proximally updates the policy, applying local updates to ensure that the new policy does not deviate excessively from the previous one. This prevents large updates that could result in unstable behavior.

The original model (depicted in Fig. 1) utilized a `4-feature` input architecture, comprising the following.

1) An `RGB` image of the environment, obtained by concatenating three $90 \times 120 \times 3$ images: one from the left-side camera, one from the centre camera, and one from the right-side camera.
2) A 9-dimensional array `G` encoding road features. This vector contains information, such as whether the vehicle is at an intersection, near a traffic light, the state of the traffic light, and the maximum speed limit.
3) A 4-dimensional array `V` incorporating vehicle features, including the current vehicle speed, real-time values of the throttle and brake, and the similarity score of the vehicle compared to the next waypoint of the planned route (centre of a road segment).
4) An array `N` of dimension $n = 5$ containing navigation features, such as distances between the actual vehicle position and the next n waypoints of the planned route.

The agent operates using a DNN architecture, receiving the current observation as input and producing the next action based on a predefined policy. This network consists of two main components: 1) the policy branch (actor) and 2) the value branch (critic), each with its own set of parameters. The policy branch selects actions from a *Beta* distribution, while the value branch estimates the value of states to compute the advantage function using the GAE technique.

Both branches share a common neural network responsible for processing observations into an intermediate representation. This network is applied sequentially to each component of the observation tensor, resulting in four intermediate representations that are then aggregated using gated recurrent units (GRUs). Additionally, the network incorporates a ShuffleNet V2 [46] backbone for processing image data.

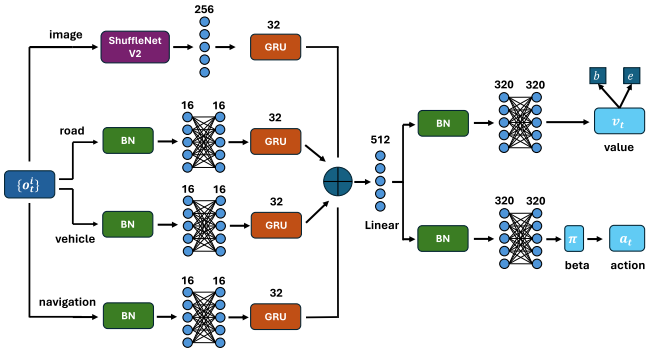

Fig. 1. DNN architecture proposed by Anzalone et al. [21].

The policy and value branches are feed-forward neural networks with two layers, comprising 320 `units` activated by Sigmoid Linear Unit (`SiLU`) and batch normalisation. The overall architecture includes fully connected layers, with concatenation along the first dimension and independent aggregation via GRU layers. Finally, values are decomposed into bases and exponents for further analysis [21].

B. Segmentation Camera Integration

Starting from the original architecture described in the previous section [21], an experimental investigation into applying environment segmentation was proposed. This exploration aimed to assess the potential enhancement of the model's performance through semantic segmentation integration.

Image semantic segmentation is a fundamental process in computer vision, crucial for various mid-level applications. It involves partitioning an image into multiple coherent regions or segments, each representing a distinct object or area of interest [47]. Formally, given an image domain D, its partition in different segments S_i can be defined as follows:

$$D = \bigcup_i S_i, \ \{S_i \cap S_j = \varnothing \ \forall i \neq j\}. \tag{3}$$

This task enables computers to comprehend and interpret image contents by identifying and classifying different regions of pixels based on visual characteristics. Image segmentation plays a pivotal role in tasks, such as object detection, image recognition, and scene understanding.

Initially, segmented information was integrated alongside the `RGB` camera view, necessitating the addition of a second backbone (ShuffleNet V2) for processing the segmented image separately. The embeddings from the second ShuffleNet were concatenated with the rest of the inputs and processed according to the architecture structure shown in Fig. 2.

To address the increased computational load and training time resulting from the presence of the second ShuffleNet, this enhanced architecture was, preliminary, evaluated on the first stage (Stage 1) in order to assess the effective contribution of segmentation combined with the `RGB` camera. This evaluation involved comparing the performance of the original `RGB` camera, the `RGB` and segmentation cameras processed separately, and solely the segmentation camera.

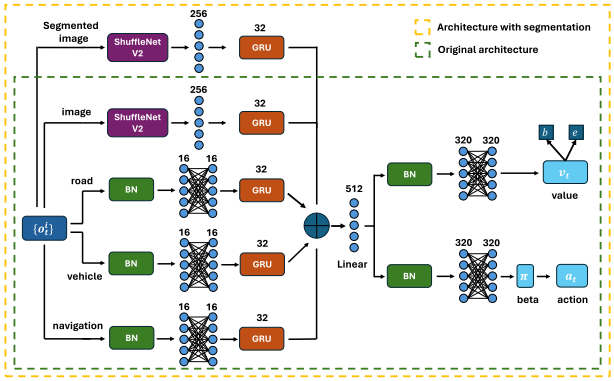

Fig. 2. Improved architecture with the addition of a second ShuffleNet V2 to process the segmented camera image.

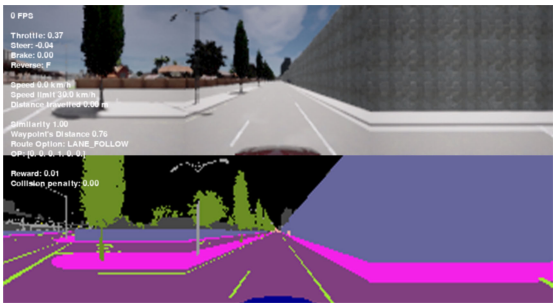

Fig. 3. Comparison between RGB camera view and segmentation camera view in the CARLA simulator environment. In the segmentation view, each object is color-coded according to its semantic class.

When considering only the segmentation camera, the model's input underwent adjustments, replacing the RGB camera with a semantic segmentation camera view. The input image dimensions remained consistent with those of the RGB camera, maintaining three color channels of size $90 \times 120 \times 3$.

Throughout the training process employing the DRL algorithm, the model learned to effectively utilize the semantic information of objects in its surroundings to make optimal decisions regarding AD tasks.

The segmentation camera provided by the CARLA environment was utilized to implement environment segmentation. This camera categorizes each object within its field of view by representing it with a distinct color based on its assigned tags. For example, pedestrians are depicted in one color while vehicles are represented in another. Upon the simulation's initiation, every element within the scene is assigned a tag, including actors during spawning [43].

The scene segmentation distinguishes 12 different objects, assigning each one a unique tag and color: *0: Unlabelled; 1: Building; 2: Fence; 3: Other; 4: Pedestrian; 5: Pole; 6: Road line; 7: Road; 8: Sidewalk; 9: Vegetation; 10: Car; 11: Wall; 12: Traffic sign*. The server provides an image with tag information encoded in the red channel, where a pixel with a red value of t corresponds to an object tagged with t. An example of a segmented camera view can be seen in Fig. 3.

C. Depth Camera Integration

An addition of spatial information was also explored to assess the performance of the CL-based approach proposed in [21]. Incorporating a depth camera provides insights into the distances of objects within the environment, enhancing the model's perception of spatial dimensions and object geometry. Moreover, it equips the system with a means to comprehend the 3-D layout of the environment and the spatial interactions between the autonomous vehicle and its surroundings. This capability proves beneficial for executing overtaking manoeuvres, navigating obstacles, or accurately gauging the distances to pedestrians and nearby vehicles. A depth camera captures depth information in addition to traditional color imagery. Unlike conventional cameras, which capture 2-D images, depth cameras measure the distance to objects in the scene, enabling the creation of depth maps or point clouds. Depth cameras utilize various technologies, such as time-of-flight (ToF) sensors, structured light, or stereo vision, to accurately measure depth and create a 3-D representation of the scene. These cameras find applications in fields, such as computer vision, robotics, augmented reality, and gesture recognition, where precise depth information is crucial for tasks, such as object recognition, urban scene understanding, and spatial mapping [48].

Following a similar procedure to the segmentation camera integration process (Section IV-B), the depth input camera available in the CARLA environment was used to collect spatial data.

The depth camera in CARLA operates by providing raw scene data, which encodes the distance of each pixel from the camera ($z - buffer$) to generate a depth map of the scene.

This depth information is encoded per pixel using three channels of the RGB color space, with the R, G, and B channels representing the least to most significant bytes, respectively. To decode the actual distance in meters, the following formula is applied:

$$\text{normalisation} = \frac{\left(R + G * 256 + B * 256^2\right)}{\left(256^3 - 1\right)} \tag{4}$$

$$\text{dist_in_meters} = 1000 * \text{normalisation}. \tag{5}$$

In this way a grayscale image is obtained. Two options are available in CARLA to obtain a depth view: 1) depth, and 2) logarithmic depth. While both provide millimetric precision, the logarithmic approach yields better results for closer objects.

D. Fusion Approach

To evaluate the agent's performance with enriched perceptual input, depth data were concatenated with the camera view (either RGB or segmented) using an early fusion approach. This fusion method integrates depth and visual information at the initial stage, enabling effective processing of the unified sensor data.

Let $\mathbf{I}_{\text{vis}} \in \mathbb{R}^{H \times W \times 3}$ denote the visual input, which can be either the RGB or the segmented image, and let $\mathbf{I}_{\text{depth}} \in \mathbb{R}^{H \times W \times 1}$ represent the corresponding depth map. The fused

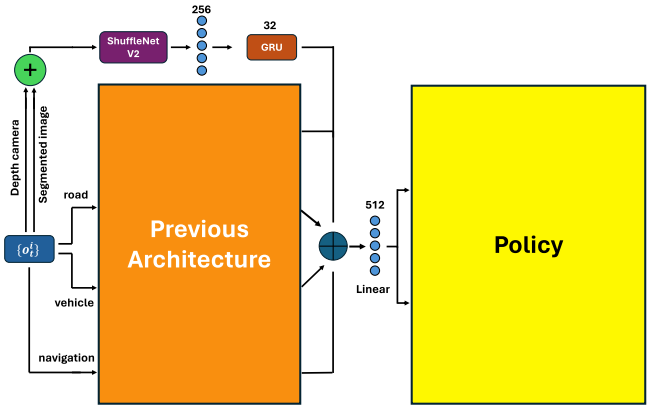

Fig. 4. Model's input configuration integrating RGB/segmented image with depth camera information.

input tensor $\mathbf{I}_{\text{fused}} \in \mathbb{R}^{H\times W\times 4}$ is obtained through channel-wise concatenation

$$\mathbf{I}_{\text{fused}} = \text{Concat}\big(\mathbf{I}_{\text{vis}}, \mathbf{I}_{\text{depth}}\big). \tag{6}$$

The fused input is subsequently processed by the ShuffleNet V2 backbone, as illustrated in Fig. 4. For consistency and comparability with the previous configuration, the network architecture remains unchanged, with modifications limited solely to the input data structure.

E. Evaluation Metrics

During the evaluation, 50 trials were conducted for each city, with each trial performed on a different road segment. The same metrics as outlined in the original paper [21] were analysed. In addition to these, Energy Consumption was included to provide insights into the environmental impact and efficiency of the system.

1) *Collision Rate:* This metric indicates the frequency of the vehicle colliding with an obstacle during a trial. It is calculated using the formula

$$\text{collision_rate} = \frac{\sum_{i=0}^{NT} c_i}{NT} \tag{7}$$

where c_i is a boolean variable representing whether the vehicle collides ($c_i = 1$) or not ($c_i = 0$) at trial i and NT is the total number of trials.

2) *Waypoint Distance:* Average distance to the next waypoint (the centre of the next road segment), expressed in meters.

3) *Similarity:* It indicates the correctness of the vehicle's current route direction relative to the next waypoint. A similarity value of 1 signifies that the vehicle is moving exactly in the correct direction, while lower values indicate deviations from the intended path.

4) *Total Reward:* This metric represents the overall reward achieved by the model during the trial.

5) *Timesteps:* The number of steps the vehicle takes during a trial. In this case, a complete trial consists of 512 timesteps. If the vehicle collides with an object, the trial is interrupted, and the number of timesteps will be less than 512.

6) *Energy Consumption:* This metric measures the energy consumed during each trial, expressed in kilowatt-hours (kWh). Energy usage was tracked using CodeCarbon [49], an open-source tool designed to monitor and quantify the environmental impact of machine learning projects in terms of carbon emissions. The energy consumption of key hardware components, including random access memory (RAM), central processing unit (CPU), and graphics processing unit (GPU), was measured to provide an accurate evaluation of the system's energy demands.

V. Experiments

The experiments were conducted using a Dell Precision 5820 tower X-Series workstation, equipped with an Intel Core i9-10980XE CPU @3.00-GHz processor, 128-GB RAM, and two NVIDIA Quadro RTX 6000 GPUs.

AD simulations were performed using CARLA simulator `v.0.9.9`.

The software implementation of the agent's model and the CL approach was developed using `Python 3.7` and `TensorFlow 2.3.1`. To ensure a fair comparison with the work of Anzalone et al. [21], the same versions of CARLA and `TensorFlow`, along with the network's hyperparameters, were maintained. This approach was taken to enable a direct benchmarking of the proposed method against the previous iteration and to accurately assess any performance improvements. The code for the original work is accessible on GitHub at the following repository: https://github.com/Luca96/carla-driving-rl-agent.

During the training phase, each of the five stages (Section IV-A) was executed for 500 episodes, divided into 512 timesteps. The network hyperparameters for both the training and testing processes were set as follows: batch size = 64, gamma = 0.9999, lambda = 0.999, policy learning rate = $3e^{-4}$, value learning rate = $3e^{-4}$, dynamics learning rate = $3e^{-4}$.

The scenarios considered are the following.

1) Integration of RGB images and segmented images (Fig. 2).
2) Use of segmentation camera only.
3) Use of depth camera only.
4) Integration of depth camera information with segmentation and RGB image (Fig. 4).

More in detail, the performances of five different agents have been compared based on the input image configuration.

1) `RGB`: RGB camera only (standard agent from [21]).
2) `RGB+D`: RGB with the integration of depth camera view.
3) `S`: segmentation camera only.
4) `D`: depth camera only.
5) `S+D`: segmentation camera with the integration of depth camera view.

For each scenario, the agent's performance was assessed using five metrics (Section IV-E) across all CARLA towns, under various weather and traffic conditions. The agent was trained exclusively in *Town03* and evaluated in *Town01*, *Town02*, *Town03*, *Town04*, *Town05*, *Town06*, *Town07*, and *Town10*. The evaluation employed two distinct sets of

TABLE I
PRELIMINARY EVALUATION APPLIED ON STAGE1 CONSIDERING ONLY RGB CAMERA (RGB), SEGMENTATION CAMERA INPUT (S) AND THEIR COMBINATION (RGB+S) EMPLOYING TWO SEPARATE BACKBONES FOR INPUT-PROCESSING. THE BEST AND THE MOST SIGNIFICANT RESULTS ARE HIGHLIGHTED IN BOLD

Metric	Agent	Town01	Town02	Town03	Town04	Town05	Town06	Town07	Town10	**Total**
Collision rate	RGB	0.70	0.80	0.80	0.44	0.14	0.18	0.16	0.14	42%
	S	0.74	0.84	0.80	0.50	0.24	**0.16**	**0.12**	0.18	45%
	RGB+S	0.76	0.84	**0.76**	0.52	0.22	0.22	0.24	0.18	47%
Similarity	RGB	0.99	0.96	0.96	0.96	0.94	0.94	0.94	0.94	95%
	S	0.99	**0.97**	**0.97**	0.96	0.94	0.94	0.94	0.94	**96%**
	RGB+S	0.99	0.96	0.95	0.96	0.94	0.94	0.94	0.94	95%
Waypoint distance	RGB	1.14	1.43	1.48	2.95	3.10	3.15	3.05	3.13	2.43 m
	S	**1.09**	1.45	**1.30**	**2.92**	3.17	**3.06**	3.15	**3.05**	**2.40 m**
	RGB+S	1.14	**1.42**	1.61	**2.92**	**3.08**	**3.04**	3.11	**3.06**	**2.42 m**
Total reward	RGB	3045.83	2753.43	2572.83	2579.53	2838.67	2779.76	2788.60	2797.53	2769.52
	S	2738.01	2491.21	**2622.63**	**2603.02**	2717.03	**2793.84**	**2799.56**	**2865.32**	2703.82
	RGB+S	2661.38	2565.80	**2729.76**	2445.19	2789.59	2765.80	2773.01	**2829.01**	2694.94
Timesteps	RGB	374.54	361.26	344.84	439.30	495.54	493.00	485.38	495.02	436.10
	S	336.78	335.76	336.58	**440.18**	483.88	488.54	**495.58**	491.92	426.15
	RGB+S	339.44	338.74	**370.60**	423.04	484.76	480.98	485.28	491.96	426.85

weather conditions. The first set (outlined in Section IV-E) was used solely for the training phase, while the second set was novel to the agent and included the following conditions: `[WetCloudySunset, WetCloudyNoon, MidRainyNoon, MidRainSunset, HardRainNoon, CloudySunset]`. Additionally, three different traffic scenarios were considered: `no traffic` (without any pedestrians or vehicles), `regular traffic` (50 pedestrians and 50 vehicles), and `dense traffic` (200 pedestrians and 100 vehicles).

VI. RESULTS AND DISCUSSION

As mentioned in Section IV-B, a preliminary evaluation was conducted during the first stage to assess the impact of segmentation on the agent's behavior. This first set of experiments aimed to determine whether segmentation enhances the agent's performance. The results for each evaluation metric are reported in Table I.

From the results shown in Table I, it can be observed that segmentation did not initially yield remarkable improvements. This behavior can be ascribed to the fact that in Stage 1, the model is not exposed to diverse traffic conditions, such as vehicles or pedestrians, which are essential for segmentation to demonstrate its advantages. The primary advantage of object semantic segmentation lies in its ability to enhance the agent's understanding of the scene and differentiate between different object categories, like vehicles, pedestrians, and other road elements. Since these objects were largely absent in Stage 1, the benefits of segmentation were limited at this early stage. Despite the lack of immediate improvement, segmentation was not abandoned, and training was extended through all five stages, where the model encountered increasingly complex traffic scenarios. Further analysis of Table I reveals that significant improvements were not achieved when both segmented and nonsegmented road images were utilized as input. However, there was a significant increase in computational time. When agents trained on either segmented (`S`) or nonsegmented (`RGB`) images individually, each epoch took approximately 1400 s. In contrast, using both segmented and nonsegmented images (`RGB+S`) resulted in training times of around 4700 seconds per epoch. Incorporation of both `RGB` and segmented images processed with two separate ShuffleNet V2 architectures substantially impacted the model, resulting in training times that were over three times slower. Given the substantial computational overhead without corresponding performance improvements, the dual-input approach was excluded. For the subsequent experiments, the `RGB` camera was replaced with a semantic segmentation camera view, allowing the model to focus on leveraging segmentation for scene understanding without the additional computational cost.

In subsequent tests, the performance of different agents trained using DRL with PPO and CL approaches, based on various input image configurations, has been analysed (see Table II). As briefly mentioned in Section V, four different input combinations have been considered: the original configuration [21] (`RGB`), the segmentation camera input (`S`), the depth camera (`D`), the original configuration with depth camera information (`RGB+D`), and the segmented input images with depth camera (`S+D`). A complete training involving all five stages was conducted for all these agents, which were trained under various conditions ranging from simple to more complex scenarios with intense traffic, numerous pedestrians, and random weather. The same training conditions and configurations as the original model were maintained to ensure a fair comparison. The obtained results are outlined in Table II. Unlike the previous work, speed results were not reported, as it was preferred to maintain a low number of frames per second (FPS) during the training process due to computational constraints. By keeping the FPS in the range of 1 to 3, surpassing the average result of 9 km/h reported by Anzalone et al. [21] was not feasible. The (`S+D`) agent demonstrates superior performance compared to the `RGB` across almost all the metrics. Notably, in cities *Town01, Town03, Town05, Town06*, and *Town07*, there is a significant reduction in the collision rate. Specifically, it records a rate of 0.29 in *Town06* and 0.31 in *Town05*, considerably lower than the corresponding rates of 0.33 and 0.49 achieved by the `RGB` agent in the same towns. This underscores the effectiveness of integrating segmentation data with depth camera information,

TABLE II
PERFORMANCE OF DIFFERENT AGENTS TRAINED USING DRL WITH PPO AND CL APPROACH BASED ON THE INPUT IMAGE CONFIGURATION: RGB CAMERA (RGB) [21], SEGMENTATION CAMERA (S), DEPTH CAMERA (D), RGB CAMERA + DEPTH CAMERA (RGB+D), SEGMENTATION CAMERA + DEPTH CAMERA,(S+D). THE BEST RESULTS ARE HIGHLIGHTED IN BOLD

Metric	Agent	Town01	Town02	Town03	Town04	Town05	Town06	Town07	Town10	Total
Collision rate	RGB	0.86	0.78	0.88	0.51	0.49	0.33	0.77	0.48	64%
	S	0.80	0.83	0.81	0.44	0.39	0.44	0.43	0.41	57%
	D	**0.73**	0.78	0.78	0.45	0.37	0.38	0.36	0.38	53%
	RGB+D	0.76	0.8	0.77	0.47	0.41	0.36	0.34	**0.36**	53%
	S+D	0.74	**0.7**	**0.74**	**0.44**	**0.31**	**0.29**	**0.35**	0.54	**51%**
Similarity	RGB	0.95	0.95	0.94	0.92	0.91	**0.96**	0.89	0.85	92%
	S	0.97	**0.96**	**0.96**	0.93	0.93	0.93	0.91	0.92	92%
	D	0.97	0.95	0.95	0.93	0.92	0.92	0.93	0.92	94%
	RGB+D	0.96	0.94	**0.96**	**0.94**	**0.93**	0.92	0.91	0.92	93%
	S+D	**0.98**	**0.96**	0.94	0.93	0.92	0.93	**0.93**	**0.94**	**94%**
Waypoint distance	RGB	1.54	1.44	1.75	3.75	3.74	5.16	2.18	4.3	2.98 m
	S	1.24	1.29	1.38	3.73	3.33	3.26	3.3	3.3	2.6 m
	D	1.31	1.5	1.39	3.75	3.31	3.34	3.24	3.3	2.64 m
	RGB+D	1.32	1.48	**1.34**	3.69	3.25	3.55	3.48	3.46	2.68 m
	S+D	**1.13**	**1.41**	1.55	**3.59**	**3.26**	**3.3**	**3.2**	**2.31**	**2.47 m**
Total reward	RGB	1866	2530	2157	2161	1764	1951	1813	1961	2025
	S	2310	2353	2364	2223	2349	2228	2184	2297	2288
	D	2570	2615	2388	2219	2377	2329	2385	2296	2397
	RGB+D	2304	2296	2381	2200	2249	2314	2411	2278	2350
	S+D	**2527**	**2797**	**2592**	**2331**	**2446**	**2426**	**2419**	**2540**	**2510**
Timesteps	RGB	296	335	347	413	406	**468**	323	400	374
	S	300	320	323	**437**	443	427	426	**439**	390
	D	321	352	325	431	445	446	443	438	400
	RGB+D	301	326	325	434	434	441	**446**	434	396
	S+D	**322**	**369**	**350**	433	**450**	455	**446**	400	**403**

enhancing the agent's ability to detect and avoid obstacles. However, a slight performance decline is observed in *Town10*, where the collision rate for S+D rises to 0.54, compared to 0.48 for RGB. Given the variability in evaluation conditions, this observation alone does not imply the inferiority of S+D. Furthermore, the total reward registered for the S+D agent outperforms all other agents across every town, achieving an overall mean value of 2510 compared to 2025 for the original RGB agent. These results highlight that the agent's behavior improves by integrating semantic segmentation and spatial data from the depth camera. The addition of depth camera information has been evaluated for both the standard RGB input image and the segmented image to assess the real contribution of the new data integration. Although the depth camera enhances the performance of the RGB agent, the total reward of the RGB+D agent (e.g., 2304 in *Town01* and 2249 in *Town05*) remains consistently lower than that of the S agent (2310 in *Town01* and 2349 in *Town05*). This outcome shows that the segmentation agent achieves superior performance even with less input data, particularly in *Town01, Town02, Town04, Town05*, and *Town06*. The experiments using only the depth camera, without any RGB input, have shown that the depth camera alone yields results comparable to those achieved with the RGB+D combination. For instance, both the D and RGB+D agents attain a total collision rate of 53%. This suggests that the contribution of RGB data is minimal compared to the depth data, potentially rendering the RGB+D combination redundant.

Furthermore, while it may seem intuitive that more information leads to better results, the reported experimentation suggests that the contribution of segmentation and depth camera (alone or in combination) outweighs that of the solely RGB. Moreover, compared to the depth camera, the significance of segmentation appears to be superior in enhancing performance. Thus, the combination of both (S+D agent) emerges as the most effective approach.

The energy usage was calculated for four distinct input image configurations: RGB camera (RGB), segmentation camera (S), depth camera (D), and a combination of segmentation and depth cameras (S+D). These evaluations were conducted in a medium-complexity scenario using *Town03*, under `regular traffic` and variable weather conditions. The reported energy consumption for the different configurations ranged from 0.753×10^{-2} kWh to 0.931×10^{-2} kWh per episode, with each episode consisting of 512 timestamps. As shown in Table III, the RGB configuration exhibited the highest total energy consumption (0.931×10^{-2} kWh), primarily due to the combined demands on the RAM, CPU, and GPU, which are driven by the processing requirements for high-resolution color images. In contrast, the segmentation camera configuration (S) consumed the least total energy at 0.753×10^{-2} kWh, likely due to the simplified visual input provided by segmented images. The results indicate that the segmentation and depth configurations offer more energy-efficient alternatives, with the S+D combination (0.850×10^{-2} kWh) providing a balanced compromise between energy consumption and data richness. These insights can further guide the selection of input configurations based on the specific energy constraints and performance requirements of the application scenario.

The results obtained in this study have potential for significant practical applications. Specifically, the research offers valuable insights into the most effective sensor combinations for improving the performance of AD systems. Additionally,

TABLE III
ENERGY CONSUMPTION IN KWH OF DIFFERENT AGENTS BASED ON FOUR DISTINCT INPUT IMAGE CONFIGURATIONS: RGB CAMERA (RGB), SEGMENTATION CAMERA (S), DEPTH CAMERA (D), AND SEGMENTATION CAMERA + DEPTH CAMERA (S+D). THE EVALUATIONS ARE CONDUCTED IN A MEDIUM-COMPLEXITY SCENARIO (*Town03*) WITH REGULAR TRAFFIC CONDITIONS. THE RESULTS HIGHLIGHT THE ENERGY DEMANDS OF EACH CONFIGURATION UNDER TYPICAL OPERATING CONDITIONS FOR A SINGLE EPISODE IN THE TESTING PHASE

Agent	RAM Energy (kWh)	CPU Energy (kWh)	GPU Energy (kWh)	Total Energy (kWh)
RGB	0.210×10^{-2}	0.364×10^{-2}	0.357×10^{-2}	0.931×10^{-2}
S	0.039×10^{-2}	0.330×10^{-2}	0.231×10^{-2}	0.753×10^{-2}
D	0.207×10^{-2}	0.357×10^{-2}	0.294×10^{-2}	0.857×10^{-2}
S+D	0.218×10^{-2}	0.375×10^{-2}	0.257×10^{-2}	0.850×10^{-2}

these findings are easily reproducible, as they were developed using widely accessible tools, such as TensorFlow and the CARLA simulator, making them applicable for practical use.

To the best of our knowledge, in addition to the comparison with Anazalone et al. [21], we identified another relevant study [50], in which the testing environment and metrics are reasonably comparable to ours. While this study reports superior performance in terms of Collision rate and Waypoint distance, it is worth noting that our approach utilizes more than triple the number of timesteps, affecting both test duration and travel distance. Given that [50] does not provide vehicle velocity data, fewer timesteps could inherently lower collision probability by reducing travel distance. To facilitate a more direct comparison, we conducted additional tests using our optimal sensor configuration, limiting the timesteps to 130. The results showed that our method achieved an almost zero Collision rate across all towns and a Waypoint distance comparable to that reported in [50].

VII. CONCLUSION

Based on the analysis of various techniques in AD and recognising the potential of DRL and CV in this domain, the study builds upon the framework proposed by Anzalone et al., in [21].

Contributions encompass the integration of segmented cameras and depth cameras into the training process alongside the standard vehicle view. Segmentation enables more precise localisation of potential obstacles, thereby enriching the agent's understanding of the surrounding environment. Similarly, the integration of depth camera data enhances the agent's ability to perceive object proximity, adding another layer of depth to its environmental perception.

Different combined approaches have been explored and evaluated to analyse the effective contribution of segmentation and depth cameras. This comprehensive evaluation involves assessing the performance of all agents using six metrics (collision rate, similarity, speed, waypoint distance, total reward, and timesteps) across different weather conditions and scenarios.

The results highlight the superiority of the agent that fuses segmented and depth camera view (S+D) over the standard agent (RGB) across multiple metrics. Notably, the S+D agent achieves a significant reduction of 13% in the collision rate and an increase in total reward from 2025 to 2510. The integration of segmented and depth cameras proves to be a fruitful strategy for enhancing the performance of AD models. By leveraging these technologies, the vehicle's ability to navigate urban environments and interact with road infrastructure is enhanced, contributing to advancements in AIoT-enabled AD.

Future research will focus on optimising data fusion strategies to enhance system performance and control computational complexity. In this context, integrating additional lightweight sensing modalities, such as event-based cameras, thermal imaging, IMUs and acoustic sensors, will be considered. These inputs can provide complementary information at a low computational and energy cost, thus supporting real-time deployment in scenarios with resource constraints. Fusion approaches beyond simple concatenation will be explored. In particular, mid-level fusion mechanisms will be investigated to support adaptive and context-aware integration of sensory data, enabling the agent to dynamically prioritize the most relevant information based on the operational environment. Furthermore, techniques such as neural network quantisation will be evaluated to mitigate the computational burden introduced by multimodal architectures without compromising policy performance.

Finally, fault detection and isolation (FDI) algorithms will be implemented to ensure robust system behavior in the presence of sensor faults or degraded inputs [51]. These advancements aim to improve the reliability, scalability and efficiency of the proposed framework further, thus facilitating its adoption in real-world AIoT-enabled AD systems.

REFERENCES

[1] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda, "A survey of autonomous driving: *Common practices and emerging technologies*," *IEEE Access*, vol. 8, pp. 58443–58469, 2020.

[2] L. Liu et al., "Computing systems for autonomous driving: State of the art and challenges," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6469–6486, Apr. 2020.

[3] B. Wang, X. Gong, Y. Wang, P. Lyu, and S. Liang, "Coordination for connected and autonomous vehicles at unsignalized intersections: An iterative learning based collision-free motion planning method," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 5439–5454, Feb. 2024.

[4] Y. Li, M. Liu, Q. Yang, Z. Shen, and W. Wu, "Collision-free autonomous scheduling at unsignalized intersection using conflict graph tree search," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14563–14578, Apr. 2024.

[5] I. Ud Din, A. Almogren, and J. J. P. C. Rodrigues, "AIoT integration in autonomous vehicles: Enhancing road cooperation and traffic management," *IEEE Internet Things J.*, vol. 11, no. 22, pp. 35942–35949, Nov. 2024.

[6] D. Fernandez-Llorca and E. Gómez, "Trustworthy artificial intelligence requirements in the autonomous driving domain," *Computer*, vol. 56, no. 2, pp. 29–39, Feb. 2023.

[7] Y. Li et al., "Choose your simulator wisely: A review on open-source simulators for autonomous driving," *IEEE Trans. Intell. Veh.*, vol. 9, no. 5, pp. 4861–4876, May 2024.

[8] Q. Chao et al., "A survey on visual traffic simulation: Models, evaluations, and applications in autonomous driving," *Comput. Graph. Forum*, vol. 39, no. 1, pp. 287–308, 2020.

[9] V. François-Lavet et al., "An introduction to deep reinforcement learning," *Found. Trends® Mach. Learn.*, vol. 11, no. 3-4, pp. 219–354, 2018.

[10] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017.

[11] B. R. Kiran et al., "Deep reinforcement learning for autonomous driving: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 4909–4926, Jun. 2022.

[12] X. Zhang, L. Wu, H. Liu, Y. Wang, H. Li, and B. Xu, "High-speed ramp merging behavior decision for autonomous vehicles based on multiagent reinforcement learning," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 22664–22672, Dec. 2023.

[13] V. Chamola, A. Chougule, A. Sam, A. Hussain, and F. R. Yu, "Overtaking mechanisms based on augmented intelligence for autonomous driving: Data sets, methods, and challenges," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17911–17933, May 2024.

[14] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. 26th Annu. Int. Conf. Mach. Learn.*, 2009, pp. 41–48.

[15] A. Ozturk et al., "Investigating value of curriculum reinforcement learning in autonomous driving under diverse road and weather conditions," in *Proc. IEEE Intell. Veh. Symp. Workshops (IV Workshops)*, 2021, pp. 358–363.

[16] Z. Qiao, K. Muelling, J. M. Dolan, P. Palanisamy, and P. Mudalige, "Automatically generated curriculum based reinforcement learning for autonomous vehicles in urban environment," in *Proc. IEEE Intell. Veh. Symp. (IV)*, 2018, pp. 1233–1238.

[17] Y. Xiao, F. Codevilla, A. Gurram, O. Urfalioglu, and A. M. López, "Multimodal end-to-end autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 1, pp. 537–547, Jan. 2020.

[18] D. Feng et al., "Deep multi-modal object detection and semantic segmentation for autonomous driving: Datasets, methods, and challenges," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 3, pp. 1341–1360, Mar. 2021.

[19] L. Wang et al., "Multi-modal 3D object detection in autonomous driving: A survey and taxonomy," *IEEE Trans. Intell. Veh.*, vol. 8, no. 7, pp. 3781–3798, Jul. 2023.

[20] W. Liu et al., "A systematic survey of control techniques and applications in connected and automated vehicles," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21892–21916, Dec. 2023.

[21] L. Anzalone, P. Barra, S. Barra, A. Castiglione, and M. Nappi, "An end-to-end curriculum learning approach for autonomous driving scenarios," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 19817–19826, Oct. 2022.

[22] K. Muhammad, A. Ullah, J. Lloret, J. Del Ser, and V. H. C. de Albuquerque, "Deep learning for safe autonomous driving: Current challenges and future directions," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4316–4336, Jul. 2021.

[23] L. Le Mero, D. Yi, M. Dianati, and A. Mouzakitis, "A survey on imitation learning techniques for end-to-end autonomous vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 14128–14147, Sep. 2022.

[24] A. O. Ly and M. Akhloufi, "Learning to drive by imitation: An overview of deep behavior cloning methods," *IEEE Trans. Intell. Veh.*, vol. 6, no. 2, pp. 195–209, Jun. 2021.

[25] A. Y. Ng and S. Russell, "Algorithms for inverse reinforcement learning," in *Proc. ICML*, 2000, p. 2.

[26] F. Codevilla, E. Santana, A. M. López, and A. Gaidon, "Exploring the limitations of behavior cloning for autonomous driving," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 9329–9338.

[27] X. Hu, B. Tang, L. Chen, S. Song, and X. Tong, "Learning a deep cascaded neural network for multiple motion commands prediction in autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 12, pp. 7585–7596, Jul. 2020.

[28] S. Teng et al., "Motion planning for autonomous driving: The state of the art and future perspectives," *IEEE Trans. Intell. Vehicles*, vol. 9, no. 6, pp. 3692–3711, May 2023.

[29] J. Chen, S. E. Li, and M. Tomizuka, "Interpretable end-to-end urban autonomous driving with latent deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 5068–5078, Jun. 2022.

[30] A. Kendall et al., "Learning to drive in a day," in *Proc. Int. Conf. Robot. Autom.*, 2019, pp. 8248–8254.

[31] R. Gutiérrez-Moreno, R. Barea, E. López-Guillén, J. Araluce, and L. M. Bergasa, "Reinforcement learning-based autonomous driving at intersections in CARLA simulator," *Sensors*, vol. 22, no. 21, p. 8373, 2022.

[32] D. Isele, R. Rahimi, A. Cosgun, K. Subramanian, and K. Fujimura, "Navigating occluded intersections with autonomous vehicles using deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2018, pp. 2034–2039.

[33] D. Isele, A. Nakhaei, and K. Fujimura, "Safe reinforcement learning on autonomous vehicles," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2018, pp. 1–6.

[34] J. Liao, T. Liu, X. Tang, X. Mu, B. Huang, and D. Cao, "Decision-making strategy on highway for autonomous vehicles using deep reinforcement learning," *IEEE Access*, vol. 8, pp. 177804–177814, 2020.

[35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[36] O. Natan and J. Miura, "End-to-end autonomous driving with semantic depth cloud mapping and multi-agent," *IEEE Trans. Intell. Veh.*, vol. 8, no. 1, pp. 557–571, Jan. 2023.

[37] Z. Huang, C. Lv, Y. Xing, and J. Wu, "Multi-modal sensor fusion-based deep neural network for end-to-end autonomous driving with scene understanding," *IEEE Sensors J.*, vol. 21, no. 10, pp. 11781–11790, May 2021.

[38] P. Jin, Y. Lin, Y. Song, T. Li, and W. Yang, "Vision-force-fused curriculum learning for robotic contact-rich assembly tasks," *Front. Neurorobot.*, vol. 17, Oct. 2023, Art. no. 1280773.

[39] G. Rizzoli, F. Barbato, and P. Zanuttigh, "Multimodal semantic segmentation in autonomous driving: A review of current approaches and future perspectives," *Technologies*, vol. 10, no. 4, p. 90, 2022.

[40] X. Zhang et al., "OpenMPD: An open multimodal perception dataset for autonomous driving," *IEEE Trans. Veh. Technol.*, vol. 71, no. 3, pp. 2437–2447, Jan. 2022.

[41] P. Testolina, F. Barbato, U. Michieli, M. Giordani, P. Zanuttigh, and M. Zorzi, "SELMA: Semantic large-scale multimodal acquisitions in variable weather, daytime and viewpoints," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 7, pp. 7012–7024, Mar. 2023.

[42] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," 2015, *arXiv:1506.02438*.

[43] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. Conf. Robot Learn.*, 2017, pp. 1–16.

[44] M. Quigley et al., "ROS: An open-source robot operating system," in *Proc. ICRA Workshop Open Source Softw.*, 2009, p. 5.

[45] M. Dupuis et al., "Opendrive 2010 and beyond–status and future of the de facto standard for the description of road networks," in *Proc. Driving Simul. Conf. Europe*, 2010, pp. 231–242.

[46] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," 2018, *arXiv:1807.11164*.

[47] P. Arbeláez, B. Hariharan, C. Gu, S. Gupta, L. Bourdev, and J. Malik, "Semantic segmentation using regions and parts," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2012, pp. 3378–3385.

[48] N. Buch, S. A. Velastin, and J. Orwell, "A review of computer vision techniques for the analysis of urban traffic," *IEEE Trans. Intell. Transp. Syst.*, vol. 12, no. 3, pp. 920–939, Sep. 2011.

[49] B. Courty et al. (Zenodo, Geneva, Switzerland). *Mlco2/Codecarbon: V2.4.1*. May 2024. [Online]. Available: https://zenodo.org/records/11171501

[50] S. Lu, L. He, S. E. Li, Y. Luo, J. Wang, and K. Li, "Hierarchical end-to-end autonomous driving: Integrating BEV perception with deep reinforcement learning," 2024, *arXiv:2409.17659*.

[51] M. Krichen, "Formal methods and validation techniques for ensuring automotive systems security," *Information*, vol. 14, no. 12, p. 666, 2023.

Silvio Barra (Member, IEEE) was born in 1985 in Salerno, Italy. He received the B.Sc. degree (cum laude) in computer science and the M.Sc. degree (cum laude) in computer science from the University of Salerno, Salerno, in 2009 and 2012, respectively, and the Ph.D. degree from the University of Cagliari, Cagliari, Italy, in 2017.

He is currently a Research Assistant with the University of Naples, Federico II, Naples, Italy. He is author in more than 60 papers, published in International Journals, conferences and books. His main research interests include pattern recognition, biometrics, video analysis, and analytics and financial forecasting.

Lucia Cimmino (Member, IEEE) received the B.S. and M.S. degrees (cum laude) in computer science from the University of Salerno, Salerno, Italy, in 2018 and 2020, respectively, and the Ph.D. degree in computer science Internet of Things and smart technologies from the Biometric and Image Processing Laboratory, University of Salerno in 2023.

She is currently a Researcher with the Department of Computer Science, University of Salerno. Her research fields include biometrics, image processing, computer vision, artificial intelligence, and sustainable computing.

Dr. Cimmino is an Associate Editor of Multimedia Tools and Applications (MTAP) for Track 2 and Track 8. She is a member of Computer Vision, Pattern Recognition and Machine Learning Italian Association. She is a member of the IEEE Signal Processing Society Young Professionals New Events and Opportunities committee.

Vincenzo Loia (Senior Member, IEEE) received the graduation degree in computer science from the University of Salerno, Salerno, Italy, in 1985, and the Ph.D. degree in computer science from Université Pierre & Marie Curie Paris VI, Paris, France, in 1989.

He is currently a Full Professor of computer science with the University of Salerno. He has authored more than 350 scientific papers, published in international conference proceedings, book chapters, and prestigious journals. His research interests include solving complex problems using new paradigms in software design and programming, including semantic technologies, Description Logic, Fuzzy Logic, data analysis for knowledge extraction, and AI-driven decision support systems based on cognitive architectures, computational intelligence, and granular computing.

Prof. Loia holds various editorial roles for numerous international journals. He is also the Founder and Editor-in-Chief of the international journal *Ambient Intelligence and Humanized Computing*.

Michele Nappi (Senior Member, IEEE) received the Laurea degree (cum laude) in computer science from the University of Salerno, Salerno, Italy, in 1991, the M.Sc. degree in information and communication technology from I.I.A.S.S. "E.R. Caianiello," Vietri, Italy, in 1997, and the Ph.D. degree in applied mathematics and computer science from the the University of Padova, Padova, Italy, in 1997.

He is currently a Full Professor of computer science with the University of Salerno. He has authored more than 250 publications. His research interest include pattern recognition, image processing, multimedia databases, biometrics, and human–computer interaction.

Dr. Nappi is an Associate Editor and Guest Editor for several international journals and is a TPC Member of numerous conferences. As the Team Leader of the Biometric and Image Processing Lab, he has received multiple international awards. He is a GIRPR/IAPR Member. He co-founded the BS3 spin-off and serves as Deputy Rector for the University of Salerno Reputation and International Ranking. He is the Co-Editor in Chief of *Image and Vision Computing* Journal and Section Editor for *Multimedia Tools and Applications* Track 8 (Medical Imaging).

Matteo Polsinelli (Member, IEEE) received the M.S. degree (cum laude) in computer engineering and the Ph.D. degree in health and environmental sciences (imaging curriculum) from the University of L'Aquila, L'Aquila, Italy, in 2018 and 2022, respectively.

He is currently a Researcher with the Department of Management and Innovation Systems, University of Salerno, Salerno, Italy. His research interests include advanced methodologies in machine learning and deep learning related to computer vision. Currently, he is focused on the detection of fake images generated through artificial intelligence systems.

Dr. Polsinelli served as the Finance Chair for the IEEE 36th International Symposium on Computer-Based Medical Systems 2023. He is an Associate Editor for *Multimedia Tools and Applications* and serves as reviewer for several international journals. He is a member of Computer Vision, Pattern Recognition and Machine Learning Italian Association.